

Core Language Working Group "ready" Issues for the July, 2022 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4910](#).

2355. Deducing *noexcept-specifiers*

Section: 13.10.3.6 [[temp.deduct.type](#)] **Status:** ready **Submitter:** John Spicer **Date:** 2017-09-06

The list of deducible forms in 13.10.3.6 [[temp.deduct.type](#)] paragraph 8 does not include the ability to deduce the value of the constant in a *noexcept-specifier*, although implementations appear to allow it.

Notes from the April, 2018 teleconference:

Although this appears to be an obvious omission, CWG felt that EWG should weigh in on whether this capability should be supported or not.

EWG guidance (January, 2021):

Modify the Standard such that the value of a constant in a *noexcept-specifier* can be deduced. See [vote](#).

Proposed resolution (June, 2022):

1. Change 13.10.3.6 [[temp.deduct.type](#)] paragraph 3 as follows:

A given type P can be composed from a number of other types, templates, and non-type values:

- A function type includes the types of each of the function parameters ~~and~~, the return type, **and its exception specification.**
- ...

2. Add the following to Example 3 in 13.10.3.6 [[temp.deduct.type](#)] paragraph 7:

Here is an example where two template arguments are deduced from a single function parameter/argument pair...

Here is an example where the exception specification of a function type is deduced:

```
template <bool E> void f1(void (*)() noexcept(E));  
template<bool> struct A { };  
template<bool B> void f2(void (*) (A<B>) noexcept(B));
```

```
void g1();  
void g2() noexcept;  
void g3(A<true>);
```

```
void h() {  
    f1(g1);    // OK: E is false  
    f1(g2);    // OK: E is true  
}
```

```
f2(g3); // error: B deduced as both true and false
}
```

Here is an example where a qualification conversion applies...

3. Change 13.10.3.6 [temp.deduct.type] paragraph 8 as follows:

A template type argument T , a template template argument TT , or a template non-type argument i can be deduced if P and A have one of the following forms:

```
 $T$ 
 $cv_{opt} T$ 
 $T^*$ 
 $T\&$ 
 $T\&\&$ 
 $T_{opt} [integer\_constant\ i_{opt}]$ 
template-name< $T$ > (where template-name refers to a class template)
type( $T$ )
 $T()$ 
 $T_{opt} (T_{opt})\ noexcept(i_{opt})$ 
 $T_{type::*}$ 
type- $T::*$ 
 $T_{opt}\ T_{opt}::*$ 
 $T (type::*)()$ 
type- $(T::*)()$ 
type- $(type::*)(T)$ 
type- $(T::*)(T)$ 
 $T (type::*)(T)$ 
 $T (T::*)(T)$ 
 $T (T::*)(T)$ 
type[ $i$ ]
template-name< $i$ > (where template-name refers to a class template)
 $TT_{opt} <T>$ 
 $TT_{opt} <i>$ 
 $TT_{opt} <TT>$ 
 $TT_{opt} <>$ 
```

where (T) represents a parameter-type-list (9.3.4.6 [dcl.fct]) where at least one parameter type contains a T , and $()$ represents a parameter-type-list where no parameter type contains a T .

- T_{opt} represents a type or parameter-type-list that either satisfies these rules recursively, is a non-deduced context in P or A , or is the same non-dependent type in P and A ,
- TT_{opt} represents either a class template or a template template parameter,
- i_{opt} represents an expression that either is an i , is value-dependent in P or A , or has the same constant value in P and A , and
- $noexcept(i_{opt})$ represents an exception specification (14.5 [except.spec]) in which the (possibly-implicit, see 9.3.4.6 [dcl.fct]) *noexcept-specifier's* operand satisfies the rules for an i_{opt} above.

[Note: If a type matches such a form but contains no T s, i s, or TT s, deduction is not possible. —end note]

Similarly, $<T>$ represents template argument lists where at least one argument contains a T , $<i>$ represents template argument lists where at least one argument contains an i and $<>$ represents template argument lists where no argument contains a T or an i .

4. Add the following as a new paragraph following 13.10.3.6 [temp.deduct.type] paragraph 14:

The type of *N* in the type *T*[*N*] is `std::size_t`. [Example 9: ... —end example]

The type of *B* in the *noexcept-specifier* `noexcept(B)` of a function type is `bool`.

[Example:

```
template<bool> struct A { };
template<auto> struct B;
template<auto X, void (*F)() noexcept(X)> struct B<F> {
    A<X> ax;
};
void f_nothrow() noexcept;
B<f_nothrow> bn;    // OK: type of X deduced as bool
```

—end example]

5. Change 13.10.3.6 [temp.deduct.type] paragraph 19 as follows:

If *P* has a form that contains `<i>`, and if the type of *i* differs from the type of the corresponding template parameter of the template named by the enclosing *simple-template-id*, deduction fails. If *P* has a form that contains `[i]`, and if the type of *i* is not an integral type, deduction fails.¹³¹ **If *P* has a form that includes `noexcept(i)` and the type of *i* is not `bool`, deduction fails.**

[Example 12: ...

6. Add the following as a new section preceding C.1.4 [diff.cpp20.library]:

C.1.4 Clause 13: templates

[diff.cpp20.temp]

Affected subclause: 13.10.3.6 [temp.deduct.type]

Change: Deducing template arguments from exception specifications.

Rationale: Facilitate generic handling of throwing and non-throwing functions.

Effect on original feature: Valid ISO C++20 code may be ill-formed in this revision of C++.

[Example 1:

```
template<bool> struct A { };
template<bool B> void f(void (*)(A<B>) noexcept(B));
void g(A<false>) noexcept;
void h() {
    f(g);    // ill-formed; previously well-formed.
}
```

—end example]

2405. Additional type-dependent expressions

Section: 13.8.3.3 [temp.dep.expr] **Status:** ready **Submitter:** Andrey Davydov **Date:** 2018-08-20

According to 13.8.3.3 [temp.dep.expr] paragraph 3,

...Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

```
simple-type-specifier ( expression-list )
::opt new new-placementopt new-type-id new-initializeropt
::opt new new-placementopt ( type-id ) new-initializeropt
dynamic_cast < type-id > ( expression )
static_cast < type-id > ( expression )
```

```
const_cast < type-id > ( expression )
reinterpret_cast < type-id > ( expression )
( type-id ) cast-expression
```

This list is missing cases for:

```
simple-type-specifier braced-init-list
typename-specifier ( expression-listopt )
typename-specifier braced-init-list
```

Proposed resolution (approved by CWG 2022-06-17):

Change in 13.8.3.3 [[temp.dep.expr](#)] paragraph 3 as follows:

Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier*, **typename-specifier**, or *new-type-id* is dependent, even if any subexpression is type-dependent:

```
simple-type-specifier ( expression-listopt )
simple-type-specifier braced-init-list
typename-specifier ( expression-listopt )
typename-specifier braced-init-list
...
```

2507. Default arguments for operator[]

Section: 12.4.1 [[over.oper.general](#)] **Status:** ready **Submitter:** Jens Maurer **Date:** 2021-12-07

The intent of paper P2128R6, which permitted multiple parameters in overloaded subscript operators and was adopted at the October, 2021 plenary, was that overloaded operator[] should allow parameters with default arguments. However, the adopted wording did not address the following restriction from 12.4.1 [[over.oper.general](#)] paragraph 10:

An operator function cannot have default arguments (9.3.4.7 [[dcl.fct.default](#)]), except where explicitly stated below.

Similar wording to that of operator() should be added for operator[].

Proposed resolution (December, 2021):

Change 12.4.5 [[over.sub](#)] paragraph 1 as follows:

A subscripting operator function is a function named operator[] that is a non-static member function **with an arbitrary number of parameters. It may have default arguments.** For an expression...

Approved by EWG 2022-04-14.

Approved by CWG 2022-04-22.

2534. Value category of pseudo-destructor expression

Section: 7.6.1.5 [[expr.ref](#)] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2022-02-17

Subclause 7.6.1.5 [[expr.ref](#)] paragraph 3 defines the value category of a pseudo-destructor class member access expression to be an lvalue:

Abbreviating *postfix-expression.id-expression* as E1.E2, E1 is called the *object expression*. If the object expression is of scalar type, E2 shall name the pseudo-destructor of that same type (ignoring cv-qualifications) and E1.E2 is an lvalue of type “function of () returning void”.

This is inconsistent with the analogous situation naming the destructor of a class. In that case, the class member access expression is a prvalue, not an lvalue, as specified in 7.6.1.5 [expr.ref] bullet 6.3 (see also issue 2458):

- If E2 is an overload set, function overload resolution (12.2 [over.match]) is used to select the function to which E2 refers. The type of E1.E2 is the type of E2 and E1.E2 refers to the function referred to by E2.
 - If E2 refers to a static member function, E1.E2 is an lvalue.
 - Otherwise (when E2 refers to a non-static member function), E1.E2 is a prvalue. The expression can be used only as the left-hand operand of a member function call (11.4.2 [class.mfct]).

It also contradicts 7.2.1 [basic.lval] bullet 1.1:

- A *glvalue* is an expression whose evaluation determines the identity of an object or function.

A pseudo-destructor does not have an identity.

Proposed resolution (approved by CWG 2022-04-08):

Change 7.6.1.5 [expr.ref] paragraph 3 as follows:

If the object expression is of scalar type, E2 shall name the pseudo-destructor of that same type (ignoring cv-qualifications) and E1.E2 is ~~an lvalue~~ **a prvalue** of type “function of () returning void”.

2535. Type punning in class member access

Section: 7.6.1.5 [expr.ref] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2022-02-17

The initialization of `j` ought to have undefined behavior, but the standard does not explicitly say so:

```
struct C { int m; };

int i = 0;
int j = reinterpret_cast<C&>(i).m; // the same as int j = i ?
```

A related case for pointer-to-member expressions is covered by 7.6.4 [expr.mptr.oper] paragraph 4:

If the dynamic type of E1 does not contain the member to which E2 refers, the behavior is undefined.

The invocation of non-static member functions is covered by 11.4.3 [class.mfct.non.static] paragraph 2:

If a non-static member function of a class `x` is called for an object that is not of type `x`, or of a type derived from `x`, the behavior is undefined.

Proposed resolution (approved by CWG 2022-06-17):

(updated according to 2022-05-20 and 2022-06-03 CWG guidance)

1. Add a new paragraph after 7.6.1.5 [expr.ref] paragraph 7:

If E2 is a non-static ~~data member or a non-static~~ member ~~function~~, the program is ill-formed if the class of which E2 is directly a member is an ambiguous base (6.5.2 [class.member.lookup]) of the naming class (11.8.3 [class.access.base]) of E2. [Note: The program is also ill-formed if the naming class is an ambiguous base of the class type of the object expression; see 11.8.3 [class.access.base]. —*end note* -- end note]

If E2 is a non-static member and the result of E1 is an object whose type is not similar (7.3.6 [conv.qual]) to the type of E1, the behavior is undefined. [Example:

```
struct A { int i; };
struct B { int j; };
struct D : A, B {};
void f() {
    D d;
    static_cast<B&>(d).j;      // OK, object expression designates the B subobject of d
    reinterpret_cast<B&>(d).j; // undefined behavior
}
```

-- end example]

2. Change in 7.6.4 [expr.mptr.oper] paragraph 4:

If the dynamic type of E1 is the same as the static type of E1, or if the result of E1 is an object whose type is not similar to the type of E1, or whose most derived object does not contain the member to which E2 refers, the behavior is undefined. Otherwise, the expression E1 is sequenced before the expression E2.

3. Remove 11.4.3 [class.mfct.non.static] paragraph 2:

If a non-static member function of a class *x* is called for an object that is not of type *x*, or of a type derived from *x*, the behavior is undefined.

2540. Unspecified interpretation of *numeric-escape-sequence*

Section: 5.13.3 [lex.ccon] Status: ready Submitter: Richard Smith Date: 2022-02-25

Subclause 5.13.3 [lex.ccon] does not specify how the characters in an *octal-escape-sequence* or *hexadecimal-escape-sequence* are interpreted to obtain the integer value *v* that is used in bullet 3.2:

- ...
- A *character-literal* with a *c-char-sequence* consisting of a single *numeric-escape-sequence* that specifies an integer value *v* has a value as follows:
- ...

Proposed resolution (approved by CWG 2022-03-11):

1. Change in 5.13.3 [lex.ccon] bullet 3.2 as follows:

- A *character-literal* with a *c-char-sequence* consisting of a single *numeric-escape-sequence* that specifies an integer value *v* has a value as follows:
 - Let *v* be the integer value represented by the octal number comprising the sequence of *octal-digits* in an *octal-escape-sequence* or by the hexadecimal number comprising the sequence of *hexadecimal-digits* in a *hexadecimal-escape-sequence*.
 - If *v* does not exceed the range of representable values of the character-literal's type, then the value is *v*.
 - ...

2. Change in 5.13.5 [lex.string] bullet 10.2 as follows:

- Each *numeric-escape-sequence* (5.13.3 [lex.ccon]) that specifies an integer value *v* contributes a single code unit with a value as follows:
 - Let *v* be the integer value represented by the octal number comprising the sequence of *octal-digits* in an *octal-escape-sequence* or by the hexadecimal number comprising the sequence of *hexadecimal-digits* in a *hexadecimal-escape-sequence*.

- If *v* does not exceed the range of representable values of the *string-literal*'s array element type, then the value is *v*.
- ...

2571. Evaluation order for subscripting

Section: 7.6.1.2 [[expr.sub](#)] **Status:** ready **Submitter:** Corentin Jabot **Date:** 2022-04-21

The specification about the relative sequencing of multiple parameters of the subscripting operator is missing. Also, issue 2507 adds support for default arguments for user-defined subscripting operators, but the sequencing of these is unspecified, too.

Suggested resolution: [SUPERSEDED]

Add a new paragraph 4 at the end of 7.6.1.2 [[expr.sub](#)]:

If the subscript operator invokes an operator function, the sequencing restrictions of the corresponding function call expression apply (12.4.5 [[over.sub](#)], 7.6.1.3 [[expr.call](#)]).

Notes from the 2022-05-20 CWG telecon:

A wording approach amending 12.2.2.3 [[over.match.oper](#)] paragraph 2 instead would be preferred.

Possible resolution (2022-05-21): [SUPERSEDED]

Change in 12.2.2.3 [[over.match.oper](#)] paragraph 2 as follows:

Therefore, the operator notation is first transformed to the equivalent function-call notation as summarized in Table 17 (where @ denotes one of the operators covered in the specified subclause). However, **except for the subscript operator (7.6.1.2 [[expr.sub](#)])**, the operands are sequenced in the order prescribed for the built-in operator (7.6 [[expr.compound](#)]).

Notes from the 2022-06-03 CWG telecon:

Repeating the function call rules for the subscript operator in 7.6.1.2 [[expr.sub](#)] instead would be preferred, to avoid any impression of a special case.

Proposed resolution (2022-06-24, amended 2022-07-15, approved by CWG 2022-07-15):

Change in 7.6.1.2 [[expr.sub](#)] paragraph 1 as follows:

A *subscript expression* is a postfix expression followed by square brackets containing a possibly empty, comma-separated list of *initializer-clauses* **which that** constitute the arguments to the subscript operator. The *postfix-expression* **and the initialization of the object parameter of any applicable subscript operator function** is sequenced before each expression in the *expression-list* **and also before any default argument. The initialization of a non-object parameter of a subscript operator function S (12.4.5 [[over.sub](#)]), including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other non-object parameter of S.**

2582. Differing member lookup from nested classes

Section: 6.5.2 [[class.member.lookup](#)] **Status:** ready **Submitter:** Jason Merrill **Date:** 2022-05-02

Consider:

```
typedef int T;
struct A {
    struct B {
        static T t;
    };
    typedef float T; // IFNDR?
};
```

Subclause 6.5.2 [[class.member.lookup](#)] paragraph 6 specifies:

The result of the search is the declaration set of $S(N, T)$. If it is an invalid set, the program is ill-formed. If it differs from the result of a search in T for N from immediately after the *class-specifier* of T , the program is ill-formed, no diagnostic required.

It is unclear whether the lookup of T inside $A::B$ is subject to the "if it differs" rule, given that the *class-specifier* of $A::B$ ends before introducing $A::T$.

Proposed resolution (approved by CWG 2022-05-06):

Change in 6.5.2 [[class.member.lookup](#)] paragraph 6 as follows:

If it differs from the result of a search in T for N ~~from immediately after the *class-specifier*~~ **in a complete-class context** of T , the program is ill-formed, no diagnostic required.

2585. Name lookup for coroutine allocation

Section: 9.5.4 [[dcl.fct.def.coroutine](#)] **Status:** ready **Submitter:** Xu Chuanqi **Date:** 2022-05-12

Consider:

```
struct Allocator;

struct resumable::promise_type {
    void* operator new(std::size_t sz, Allocator&);
    // ...
};
resumable foo() {
    co_return;
}
```

Subclause 9.5.4 [[dcl.fct.def.coroutine](#)] paragraph 9 specifies:

... The allocation function's name is looked up by searching for it in the scope of the promise type.

- If any declarations are found, overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. The lvalues $p_1 \dots p_n$ are the succeeding arguments.
- Otherwise, a search is performed in the global scope.

If no viable function is found (12.2.3 [[over.match.viable](#)]), overload resolution is performed again on a function call created by passing just the amount of space required as an argument of type `std::size_t`.

Is the example ill-formed because `resumable::promise_type` is not viable, or is the example well-formed because the global `operator new` can be used? There is implementation divergence.

See also [LLVM issue 54881](#).

Proposed resolution (approved by CWG 2022-06-17):

(updated according to 2022-05-20, 2022-06-03, and 2022-06-17 CWG guidance)

Change in 9.5.4 [[dcl.fct.def.coroutine](#)] paragraph 9 as follows:

... The allocation function's name is looked up by searching for it in the scope of the promise type.

- If **the search finds** any declarations **are found**, overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and **has is a prvalue of** type `std::size_t`. The lvalues `p1 ... pn` are the **succeeding successive** arguments. **Otherwise, a search is performed in the global scope.** If no viable function is found (12.2.3 [[over.match.viable](#)]), overload resolution is performed again on a function call created by passing just the amount of space required as **an argument a prvalue** of type `std::size_t`.
- **If the search finds no declarations, a search is performed in the global scope. Overload resolution is performed on a function call created by passing the amount of space required as a prvalue of type `std::size_t`.**

2586. Explicit object parameter for assignment and comparison

Section: 11.10.1 [[class.compare.default](#)] **Status:** ready **Submitter:** Barry Revzin **Date:** 2022-05-07

"Deducing this" allows to declare assignment and comparison operator functions as explicit object member functions.

However, such an assignment operator can never be a copy or move assignment operator, which means it always conflicts with the implicitly-defined one:

```
struct C {  
    C& operator=(this C&, C const&); // error: can't overload with the copy assignment operator  
};
```

Similarly, `operator==` or `operator<=>` can be declared with an explicit object parameter, but they cannot be defaulted:

```
struct D {  
    bool operator==(this D const&, D const&) = default; // error: not a kind of comparison that can be defaulted  
};
```

There seems to be no reason to disallow that, for people who prefer writing all of their members with explicit object parameters.

Suggested resolution:

1. Change in 11.4.6 [[class.copy.assign](#)] paragraph 1 as follows:

A user-declared copy assignment operator `X::operator=` is a non-static non-template member function of class `X` with exactly one **non-object** parameter of type `X`, `X&`, `const X&`, `volatile X&`, or `const volatile X&`.

2. Change in 11.4.6 [[class.copy.assign](#)] paragraph 3 as follows:

A user-declared move assignment operator `X::operator=` is a non-static non-template member function of class `X` with exactly one **non-object** parameter of type `X&&`, `const X&&`, `volatile X&&`, or `const volatile X&&`.

3. Change in 11.10.1 [[class.compare.default](#)] paragraph 1 as follows:

A defaulted comparison operator function (12.4.3 [[over.binary](#)]) for some class C shall be a non-template function that is

- a non-static ~~const non-volatile~~ member of C having one parameter of type const C& and either no ~~ref-qualifier or the ref-qualifier &, or~~ or friend of C and
- a friend of C having either has two parameters of type const C& or two parameters of type C, where the implicit object parameter (if any) is considered to be the first parameter.

Additional notes (May, 2022):

Forwarded to EWG with [paper issue 1235](#), by decision of the CWG chair.

Approved by EWG telecon 2022-06-09 and EWG 2022-06 electronic poll.

See [vote](#).

Additional notes (July, 2022):

The suggested resolution makes the following a copy assignment operator, suppressing the implicitly-declared one, which is surprising:

```
struct B {  
    B &operator=(this int, const B &); // copy assignment operator  
};
```

Proposed resolution (approved by CWG 2022-07-15):

1. Change in 9.5.2 [[dcl.fct.def.default](#)] paragraph 2 as follows:

~~The type T1 of an~~ An explicitly defaulted special member function ~~F~~ F1 with type T1 is allowed to differ from the **corresponding special member function F2 with** type T2 ~~it would have had if it were that would have been~~ implicitly declared, as follows:

- T1 and T2 may have differing *ref-qualifiers*;
- **if F2 has an implicit object parameter of type "reference to C", F1 may be an explicit object member function whose explicit object parameter is of type "reference to C";**
- T1 and T2 may have differing exception specifications; and
- if ~~T2~~ F2 has a **non-object** parameter of type const C&, the corresponding **non-object** parameter of ~~T1~~ F1 may be of type C&.

If T1 differs from T2 in ~~any other~~ a way **other than as allowed by the preceding rules**, then:

- if ~~F~~ F1 is an assignment operator, and the return type of T1 differs from the return type of T2 or ~~T1~~ F1's **non-object** parameter type is not a reference, the program is ill-formed;
- otherwise, if ~~F~~ F1 is explicitly defaulted on its first declaration, it is defined as deleted;
- otherwise, the program is ill-formed.

2. Change in 11.4.6 [[class.copy.assign](#)] paragraph 1 as follows:

A user-declared copy assignment operator `X::operator=` is a non-static non-template member function of class X with exactly one **non-object** parameter of type X, X&, const X&, volatile X&, or const volatile X&.

3. Change in 11.4.6 [[class.copy.assign](#)] paragraph 3 as follows:

A user-declared move assignment operator `X::operator=` is a non-static non-template member function of class X with exactly one **non-object** parameter of type X&&, const X&&, volatile X&&, or const volatile X&&.

4. Change in 11.10.1 [[class.compare.default](#)] paragraph 1 as follows:

A defaulted comparison operator function (12.4.3 [[over.binary](#)]) for some class C shall be a non-template function that is

- a non-static ~~const non-volatile~~ member of C having one parameter of type const C& and either no ~~ref-qualifier or the ref-qualifier &, or~~ or friend of C and

- a friend of C having either has two parameters of type const C& or two parameters of type C, where the implicit object parameter (if any) is considered to be the first parameter.

2594. Disallowing a global function template main

Section: 6.9.3.1 [[basic.start.main](#)] **Status:** ready **Submitter:** Jim X **Date:** 2022-06-06

Consider:

```
template<class T>
int main(T) {}
```

C++20 specified in 6.9.3.1 [[basic.start.main](#)] paragraph 2:

An implementation shall not predefine the main function. This function shall not be overloaded.

While it is unclear what "overloaded" means when multiple translation units are involved, it arguably disallowed function templates called main. This prohibition was removed with P1787R6 (Declarations and where to find them).

Proposed resolution (approved by CWG 2022-06-17):

Change in 6.9.3.1 [[basic.start.main](#)] paragraph 3 and add bullets as follows:

... A program that declares

- a variable main that belongs to the global scope, or
- ~~that declares~~ a function main that belongs to the global scope and is attached to a named module, or
- **a function template main that belongs to the global scope, or**
- ~~that declares~~ an entity named main with C language linkage (in any namespace)

is ill-formed. The name main is not otherwise reserved.

2597. Replaceable allocation and deallocation functions in the global module

Section: 10.1 [[module.unit](#)] **Status:** ready **Submitter:** Gabriel dos Reis **Date:** 2022-06-17

Subclause 10.1 [[module.unit](#)] paragraph 7 implicitly attaches the replaceable global allocation or deallocation functions to the global module. Now that extern "C++" can be used to introduce declarations in the global module, even when in the purview of a named module, the provision seems superfluous.

Proposed resolution [SUPERSEDED]:

1. Change in 6.7.5.5.1 [[basic.stc.dynamic.general](#)] paragraph 2 as follows:

The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable (17.6.3 [[new.delete](#)]). A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library (16.4.5.6 [[replacement.functions](#)]). The following allocation and deallocation functions (17.6 [[support.dynamic](#)]) are implicitly declared in global scope in each translation unit of a program **and are attached to the global module (10.1 [[module.unit](#)])**.

2. Change in 10.1 [[module.unit](#)] bullet 7.2 as follows:

- If the declaration is ...
- Otherwise, if the declaration
 - is a replaceable global allocation or deallocation function (17.6.3.2 [[new.delete.single](#)], 17.6.3.3 [[new.delete.array](#)]), or
 - is a namespace-definition with external linkage; or
 - appears within a linkage-specification (9.11 [[dcl.link](#)]);
 it is attached to the global module.
- Otherwise, ...

Additional notes (June, 2022):

Forwarded to EWG with [paper issue 1273](#), by decision of the CWG chair.

Approved by EWG telecon 2022-07-07.

Proposed resolution (approved by CWG 2022-07-15):

1. Change in 6.7.5.5.1 [[basic.stc.dynamic.general](#)] paragraph 2 as follows:

The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable (17.6.3 [[new.delete](#)]); **these are attached to the global module 10.1 [[module.unit](#)]**. A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library (16.4.5.6 [[replacement.functions](#)]). The following allocation and deallocation functions (17.6 [[support.dynamic](#)]) are implicitly declared in global scope in each translation unit of a program.

2. Change in 10.1 [[module.unit](#)] bullet 7.2 as follows:

- If the declaration is ...
- Otherwise, if the declaration
 - is a replaceable global allocation or deallocation function (17.6.3.2 [[new.delete.single](#)], 17.6.3.3 [[new.delete.array](#)]), or
 - is a namespace-definition with external linkage; or
 - appears within a linkage-specification (9.11 [[dcl.link](#)]);
 it is attached to the global module.
- Otherwise, ...

2606. static_cast from "pointer to void" does not handle similar types

Section: 7.6.1.9 [[expr.static.cast](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-06-28

Consider:

```
struct S {
    int a[5];
} s;
int (*p)[] = reinterpret_cast<int(*)[]>(&s);
int n = (*p)[0];
```

This ought to have defined behavior: a pointer to s and a pointer to s.a are pointer-interconvertible, so you should be able to navigate between them this way. But the cast as shown does not work, because the type of the pointer-interconvertible object is int[5], not int[].

Proposed resolution (approved by CWG 2022-07-01):

Change in 7.6.1.9 [[expr.static.cast](#)] paragraph 13 as follows:

... Otherwise, if the original pointer value points to an object a, and there is an object b of type **similar to** ~~(ignoring cv-qualification)~~ **T** that is pointer-interconvertible (6.8.3 [basic.compound]) with a, the result is a pointer to b. Otherwise, the pointer value is unchanged by the conversion.

2608. Omitting an empty template argument list

Section: 13.10.2 [temp.arg.explicit] **Status:** ready **Submitter:** Anoop Rana **Date:** 2022-07-03

Subclause 13.10.2 [temp.arg.explicit] paragraph 4 specifies:

Trailing template arguments that can be deduced (13.10.3 [temp.deduct]) or obtained from default *template-arguments* may be omitted from the list of explicit *template-arguments*.

[Note 1: A trailing template parameter pack (13.7.4 [temp.variadic]) not otherwise deduced will be deduced as an empty sequence of template arguments. —end note]

If all of the template arguments can be deduced, they may all be omitted; in this case, the empty template argument list <> itself may also be omitted.

The wording does not allow omitting the empty template argument list <> if all of the template arguments have been obtained from default *template-arguments*. For example:

```
template<typename T = int>
int f();

int x = f();           // ill-formed per the wording
int (*y)() = f;        // ditto
```

Proposed resolution (approved by CWG 2022-07-15):

Change in 13.10.2 [temp.arg.explicit] paragraph 4 as follows:

... If all of the template arguments can be deduced **or obtained from default *template-arguments***, they may all be omitted; in this case, the empty template argument list <> itself may also be omitted.